School of Professional Education and Executive Development

The Hong Kong Polytechnic University

SEHS4510 Integrated Study (Applied Sciences)

submitted in partial fulfillment for the award of the

Bachelor of Science (Honours) Scheme in Applied Sciences

Dynamic IPS for Spam Traffic Mitigation (Machine Learning Rule Generation)

Final Report

CHUNG CHEUK LING
24045687S

Supervisor: Dr. Ka Lok Adam Wong

Number of words: 3983

Submission date: 23-April-2026

INTEGRATED STUDY REPORT DECLARATION FORM

I, CHUNG CHEUK LING (student no.: 24045687S), hereby declare the Integrated Study report, the title of which is Dynamic IPS for Spam Traffic Mitigation (Machine Learning Rule Generation) ,

submitted on 23/4/2026 for PolyU SPEED programme Bachelor of Science (Honours) Scheme in Applied Sciences, is my work and that, to the best of my knowledge and belief, it reproduces no material previously published or written, nor the work of other students, nor generative artificial intelligence (GenAI), and nor material that has been accepted for the award of any other degree or diploma, except where due acknowledgement has been made.

Word Count: 3983 words

Signed by:

(Signature of student)

Date: 23/4/2026

Note: This declaration form must be submitted together with the ***Final Report****.*

Table of Contents

1. Abstract

This project develops a Dynamic Intrusion Prevention System (IPS) that combines machine learning-generated dynamic rules with static blacklist and whitelist rules to effectively mitigate spam traffic at the network flow level. Leveraging the CSE-CIC-IDS2018 dataset with a focus on botnet traffic as a proxy for large-scale spam activity, and cross-validating with other dataset for further training. Multiple classifiers are trained and compared, including Gaussian Naïve Bayes, Decision Tree, Random Forest, Gradient Boosting Classifier, and XGBoost. Ensemble models achieve outstanding performance with accuracy, precision, recall, and F1-score all approaching 1. After comparing the feature importance among models, Random Forest is selected for the hybrid IPS. The hybrid IPS was implemented on an AWS EC2 Ubuntu server and evaluated through simulated network attacks. Results show that the dynamic ML-enhanced rules significantly outperform static rules alone, delivering excellent spam-blocking effectiveness while maintaining high efficiency and preserving user privacy by avoiding deep packet inspection. This lightweight, interpretable solution demonstrates strong potential for practical deployment in resource-constrained environments.

2. Introduction

The swift advancement of the internet and technology has transformed the way people communicate, making digital communication methods significantly more common. There are approximately 4.83 billion email users in the world, with more than 392 billion emails sent and received globally each day, which spam rate accounting for nearly half (48%) of these daily emails (CloudHQ, 2026). Spam has become rampant and prevalent, as it wastes users' time and network resources and can even harm the recipients' assets. Spam is typically sent in bulk by unknown senders and may include commercial advertisements, phishing, scams, or malware (Ahmed et al., 2022).

To mitigate the risk posed by spam, three methods are suggested: 1) enhancing user awareness, 2) blocking through blacklists, 3) employing spam detector or anti-spam filter (Moutafis et al., 2023). Many studies have proposed that deep learning and NLP techniques can optimize detection accuracy by analyzing email context. However, these approaches often require substantial resources and can be circumvented by spammers (Olivo et al., 2024). In contrast, classifiers based on Navie Bayes and Decision Trees have proven effective and across various spam studies (Zhang, 2025). Therefore, this project proposes an intrusion prevention system that combines dynamic rules (generated via machine learning) and static rules (e.g., blacklist and whitelist) to filter spam efficiently with limited resources. These machine learning approaches, among others, can enhance the effectiveness of spam detection by learning patterns from data and improving accuracy over traditional rule-based methods alone.

3. Literature Review

The concept of leveraging ML for automated security rule generation is a long-standing approach, as demonstrated by the architecture for intelligent distributed firewalling proposed by De Santis et al. (2013). The proposed model is primarily focused on the automated generation and adaptation of rules for firewalls by analyzing network traffic and identifying attack patterns. This framework depicts how security systems can move beyond static configurations to a more dynamic defense posture.

Spam is frequently linked with more severe threats, such as phishing and botnet attacks. De Santis et al. (2013) illustrate this by analyzing anomalous traffic patterns, such as activity on non-standard ports, can help identify the control channels of a botnet. Botnets are a primary tool for attackers, responsible for distributing massive volumes of spam and launching coordinated phishing campaigns (Dada et al., 2019). Spam emails often serve as the delivery vehicle for phishing attempts to deceive users into revealing sensitive information. The research shows that ML classifiers are essential not only for identifying spam but also for recognizing the anomalous network behavior that signals broader, coordinated attacks, thereby enabling a dynamic and intelligent response.

Many studies demonstrate that two model - Naïve Bayes (NB) and Decision Tree (DT) have excellent performance in blocking spam through analyzing the email content. Zhang, C. (2025) implements a machine learning spam filter with NB, DT and Support Vector Machine models, showing each model's strength on different aspects and its precision is over 87%. Su (2025) examines the use and effectiveness of NB on spam detection by using SpamAssassin Dataset with remarkable result. It achieves 97.74% accuracy, 96.6% recall, precision 96.8% and 0.97 F1-score. These show that traditional machine learning classifiers, particularly NB and DT, have been serving as foundational techniques in spam detection owing to their computational efficiency and interpretability. The Naïve Bayes classifier is a probabilistic model grounded in Bayes' theorem. It computes the posterior probability of an email content belonging to the spam class given its features:

$$P(\text{spam} \mid \text{message}) = \frac{P(\text{message} \mid \text{spam}) \cdot P(\text{spam})}{P(\text{message})}$$

The algorithm assumes conditional independence among features (the "naïve" assumption), which although it rarely holding true in complex real-world data, enables rapid training and inference even on high-dimensional datasets. In spam-filtering contexts, NB estimates word or feature frequencies in spam versus ham corpora and has repeatedly demonstrated high accuracy (often exceeding 95 %) with minimal resource overhead (Su, 2025).

In contrast, Decision Trees is rule-based, interpretable models that recursively partition the feature according to information gain or Gini impurity criteria. Each internal node represents a decision on a single feature (e.g., "Flow Duration > threshold"), while leaf nodes assign class labels (spam/ham). DTs do not require distributional assumptions and naturally handle both categorical and numerical features, making them well-suited for network metadata. Their transparency and inference also allow direct extraction of human-readable rules for firewall integration—precisely the mechanism adopted in the present study's dynamic rule generator (Zhang, 2025).

A critical prerequisite for effective network-level spam mitigation is feature engineering. Unlike content-based filters that inspect email bodies (which raise privacy concerns and can be evaded by obfuscation), the present project operates exclusively at the network-flow level using the CSE-CIC-IDS-2018 dataset. It contains 80 features including 1 label (benign / malignant differentiate by attack type), which features are flow statistics, packet-length metrics, inter-arrival timing, protocol and flag indicators and header and window information.

These features encode temporal, volumetric, and connection-behaviour signatures typical of spam activity, such as high packet rates from botnet C&C channels or unusual port usage. By focusing on metadata rather than content, the system preserves user privacy while still accomplishing robust discrimination, directly addressing one of the project's research questions on network-level predictors of spam traffic.

The methodology is further inspired by unsupervised anomaly-detection techniques that complement supervised ML. Yang (2018) demonstrated that information entropy can quantify the uncertainty in network traffic distributions, such as source/destination IPs, ports, packet types, etc. Low-entropy flows indicate normal, predictable behaviour, while sudden spikes in entropy signal anomalous or malicious activity such as spam floods or botnet reconnaissance. By normalizing features and feeding them into an entropy-augmented SVM, Yang achieved high detection rates on large-scale cloud datasets. This work highlight that hybrid supervised–unsupervised pipelines, which entropy pre-filtering followed by ML classification can reduce false positives and adapt to evolving spam patterns. This precisely rationalizes the present study of automated feedback loop that refines rules after each simulated attack.

Spam is rarely an independent threat; it shares similar behavioural with phishing and botnet activity. Zhang et al. (2025) analyzed 1,353 URLs and showed that phishing campaigns exhibit similar network

fingerprints: elevated flow rates, anomalous flag combinations, and short-lived connections—as spam delivery vectors. Their SHAP-guided feature selection and nature-inspired hyperparameter optimization (Golden Jackal, Dandelion, etc.) achieved >98 % accuracy, reinforcing the core argument that techniques for detecting malicious traffic are interchangeable. By treating spam at the network level, the present intrusion-prevention system therefore attacks the root source rather than only filtering individual messages, delivering broader protection against the interconnected ecosystem of spam, phishing, botnet campaigns.

Finally, practical factors also indicate that lightweight machine learning is preferable to deep learning for practical deployment in resource-constrained environments. Bayrak et al. (2025) deployed DNN, CNN-LSTM and other deep models on Jetson Nano edge devices for simultaneous IDS, web-attack and SMS-spam detection, achieving 98–99 % accuracy in real time. Their results confirm that carefully chosen, lightweight AI models can deliver near-DL performance on modest hardware while consuming far less power and memory. This evidence strongly supports the project's choice of NB and DT over heavier deep-learning architectures. The hybrid static and dynamic rule system run efficiently on AWS EC2 (Ubuntu) instances or similar low-cost infrastructure typical of small organizations, while still meeting the target of 70 % improvement in spam-blocking coverage.

Overall, through analysis of relevant literature, the following conclusions can be drawn: (1) NB and DT are still viable benchmarks in terms of interpretability and effectiveness; (2) behavioral network characteristics are useful for implementing privacy-preserving attacks; (3) anomaly detection methods based on entropy measures may complement supervised machine learning approaches; and (4) ML-based pipeline systems designed to handle edges have potential. It should be noted that the conclusions made above will find their reflection in the ML-enhanced firewall system design described below.

The following research questions will be also addressed in the project:

1) Which network-level features (e.g., flow rate, IP reputation, header metadata) predict spam traffic without inspecting content?

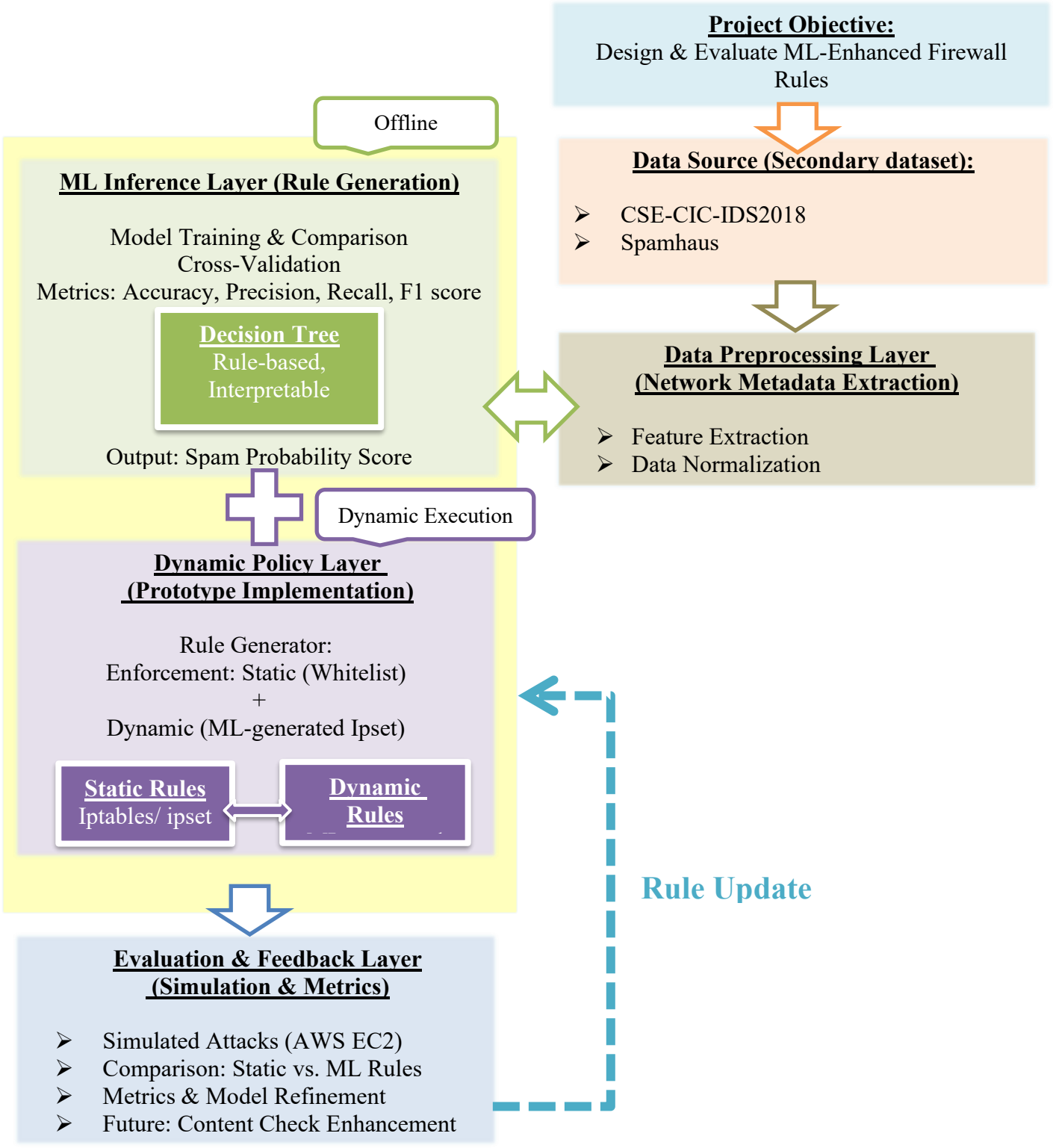
2) How much does dynamic ML-generated rule-set improve accuracy compared to a massive static blacklist?

4. Data Collection

CSE-CIC-IDS2018 is a collaboration between Communications Security Establishment (CSE) and the Canadian Institute for Cybersecurity (CIC), which is a set of comprehensive, systematic, benchmark real world datasets. The project datasets categorize 7 different attacked network scenarios in 10 days(as shown in Appendix 1): Brute-force, Heartbleed, Botnet, DoS, DDoS, Web attacks, and infiltration (Communications Security Establishment & Canadian Institute for Cybersecurity, n.d.).
All of the datasets have 79 features and 1 label (benign or type of attack) (as shown in Appendix 2), including attacks on port 25, 465, 487, 2525 and other ports, which these ports are for sending email. Since modern spam often spreads on a large scale through botnets, feature extraction of botnets is representative for identifying spam. Therefore, 03-02-2018.csv is selected for model training and cross-validation. For the remaining datasets, all malicious cases and 20% benign cases will be used to train and validate model's accuracy after prototyping the model.

5. System Design & Implementation

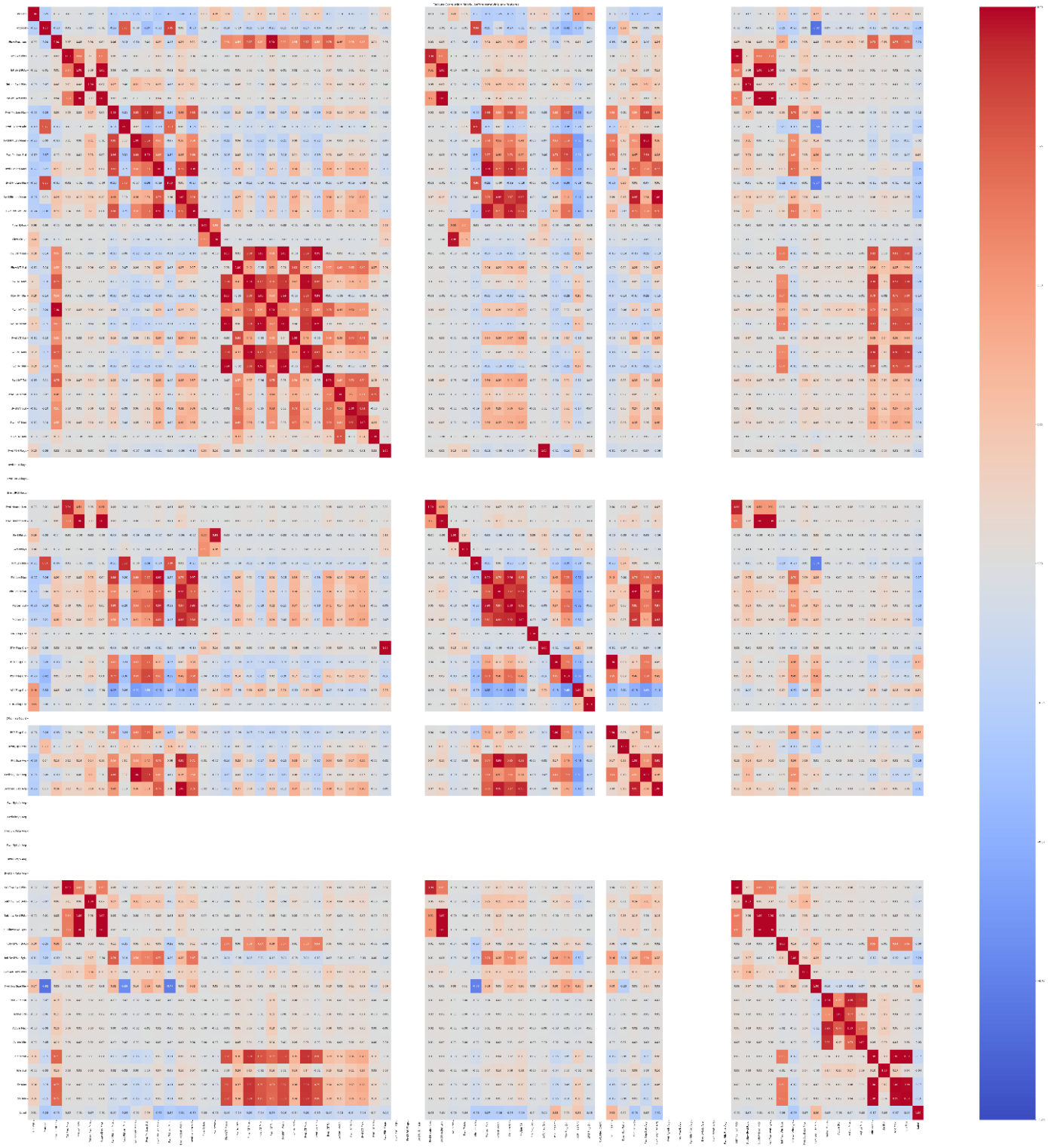
The system design starts with collecting secondary dataset CSE-CIC-IDS2018 (03-02-2018.csv) for training the model and Spamhaus IP list for static rules of the system. Then, analyzing the dataset and extracting the feature for further model training. Two classifier – Naïve Bayes and Decision Tree are trained to compare the performance for the ML inference layer. The selected model train with merged CSE-CIC-IDS2018 (all different attack flow and 20% benign flow) for enhancing the its performance and cross-validation. The final model is uploaded to a virtual server for simulation attack and refinement.



5.1 Feature Engineering

To avoid overfitting or underfitting of the model, heatmap was used to check if the features were multicollinear.

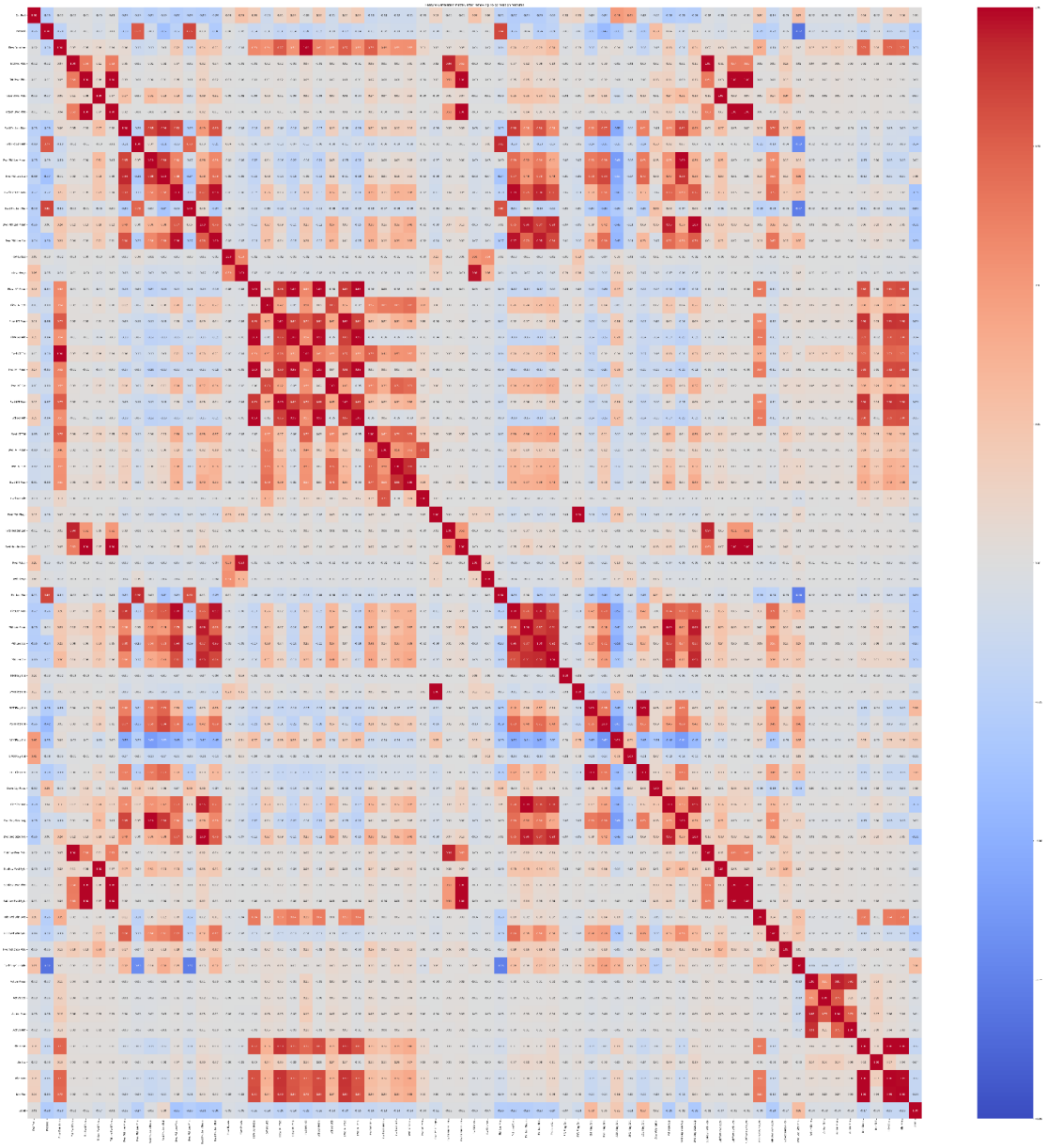
Fig.6.1 Heatmap before feature engineering:



The following features were dropped because of no correlation (white):

- Timestamp
- CWE Flag Count
- Bwd Byts/b Avg
- Bwd PSH Flags
- Fwd Byts/b Avg
- Bwd Pkts/b Avg
- Fwd URG Flags
- Fwd Pkts/b Avg
- Bwd Blk Rate Avg
- Bwd URG Flags
- Fwd Blk Rate Avg

Fig.3.2 Heatmap after dropping no correlation features:



The following group of features, (except the highest correlation with label) were dropped because they were duplicate content, which may cause multi-collinear and affect the model performance. (dark red)

Detail about Forward Paket length:

- Fwd Pkt Len Max - Fwd Pkt Len Mean - Fwd Pkt Len Std

Detail about backward Paket length

- Bwd Pkt Len Min - Bwd Pkt Len Mean - Bwd Pkt Len Std

Detail about time between two flows

- Flow IAT Mean - Flow IAT Std - Flow IAT Min

Detail about time between two packets sent in the forward direction

- Fwd IAT Mean - Fwd IAT Std - Fwd IAT Min ()

Detail about time between two packets sent in the backward direction

- Bwd IAT Mean - Bwd IAT Std - Bwd IAT Min

Detail about length of a flow

- Pkt Len Min - Pkt Len Mean - Pkt Len Std

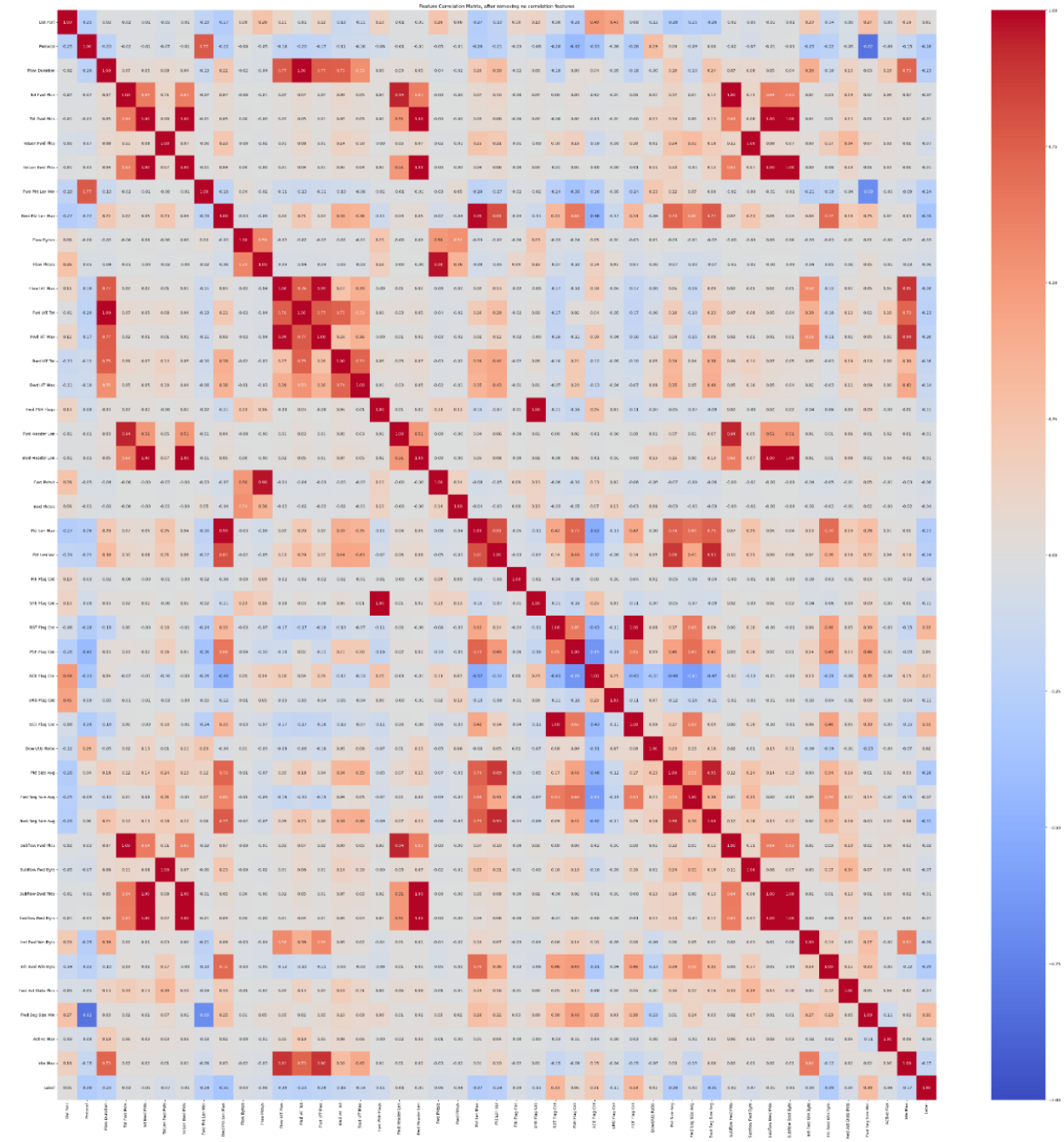
Detail about time a flow was active before becoming idle

- Active Mean - Active Std - Active Min

Detail about time a flow was idle before becoming active

- Idle Mean - Idle Std - Idle Min

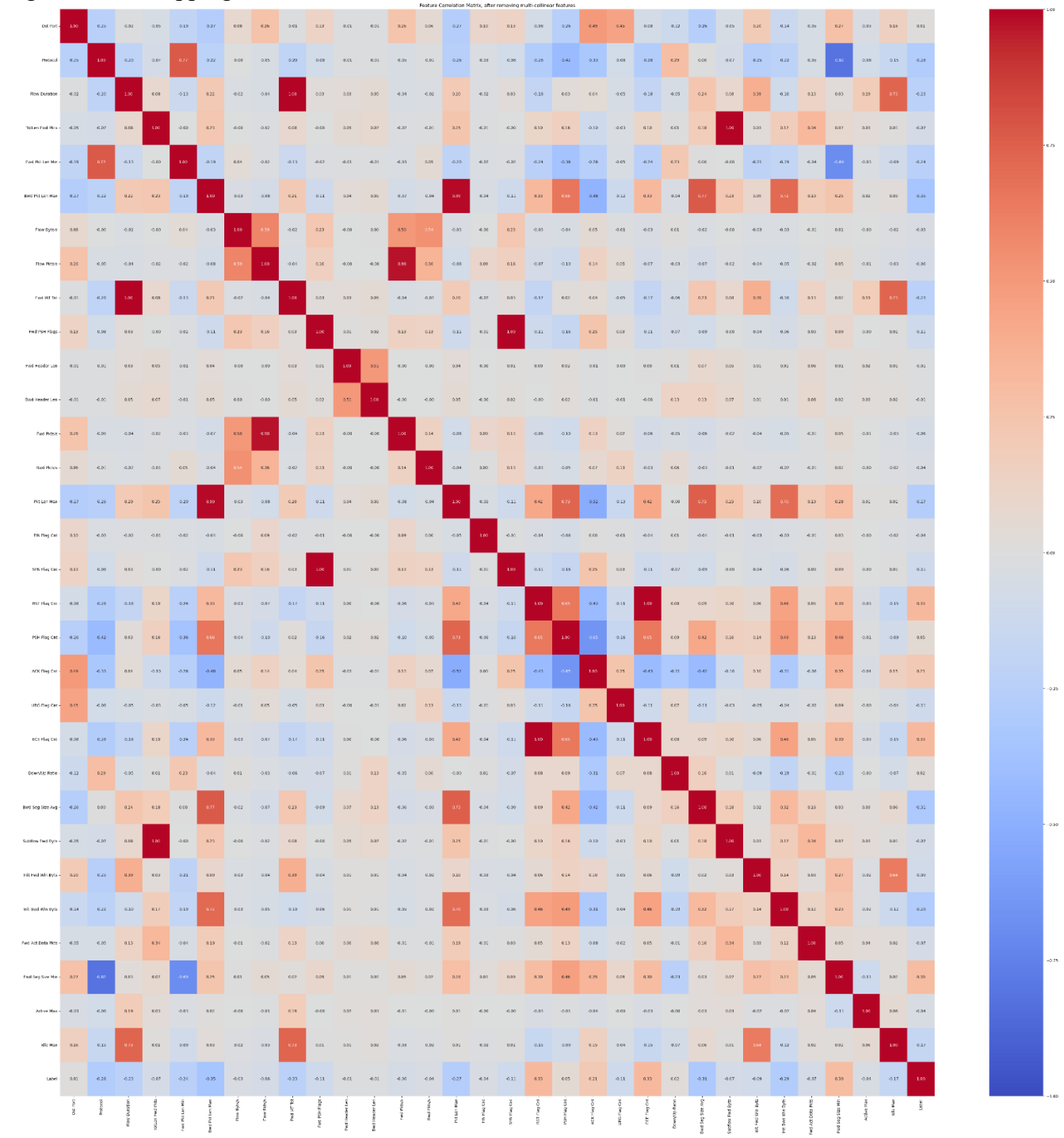
Fig.3.3 Heatmap after dropping duplicated features:



The following group of features were dropped to avoid multi-collinear. (dark red)

1. Flow IAT Max, Fwd IAT Max, Bwd IAT Tot, Bwd IAT Max (More package, longer time)
2. Tot Fwd Pkts, Tot Bwd Pkts, TotLen Bwd Pkts (More packages, longer length)
3. Pkt Size Avg, Fwd Seg Size Avg (Average packet size group)
4. Pkt Len Var group
5. Subflow feature group

Fig.3.4 After dropping multicollinear features



31 features selected for model training and categorized into 3 groups:

- Volume features: the strength and scale of the connection
- Shape features: the size and distribution of the package
- Behavioral features: the intention on network layer

These 31 features are scored by the SelectKBest for checking the correlation between the features and label. All features receive high marks except num “Fwd Header Len num” and “Bwd Header Len”(as shown in Appendix 3). However, as the header length of the packet of spam is different as legitimate normal flow, these 2 feature is kept. The final 31 features selected (as shown in Appendix 4) provide a comprehensive behavioral portrait of each network flow. They were chosen for their ability to be efficiently extracted from packet headers in real-time, without the need for deep packet inspection (DPI), which is crucial for a high-performance, network-level IPS.

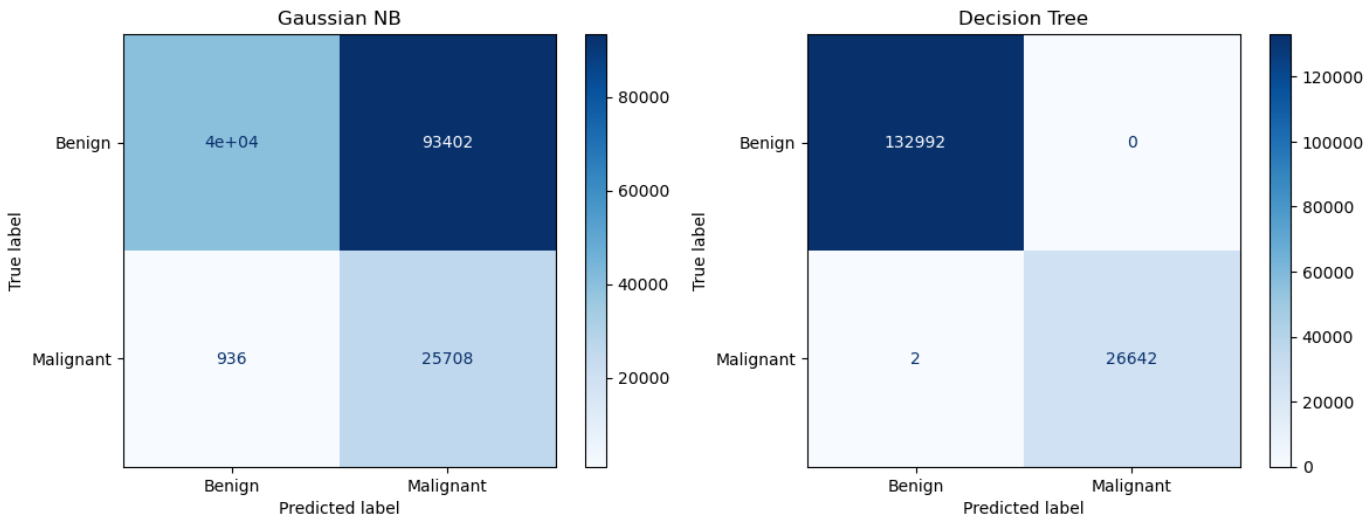
After feature engineering, constants “0” impute to the null value and infinity value in “Flow Byts/s”. In addition, Robust Scaler is used to minimize bias of the model as there are many outliers, which the outlier maybe attacks from botnet.

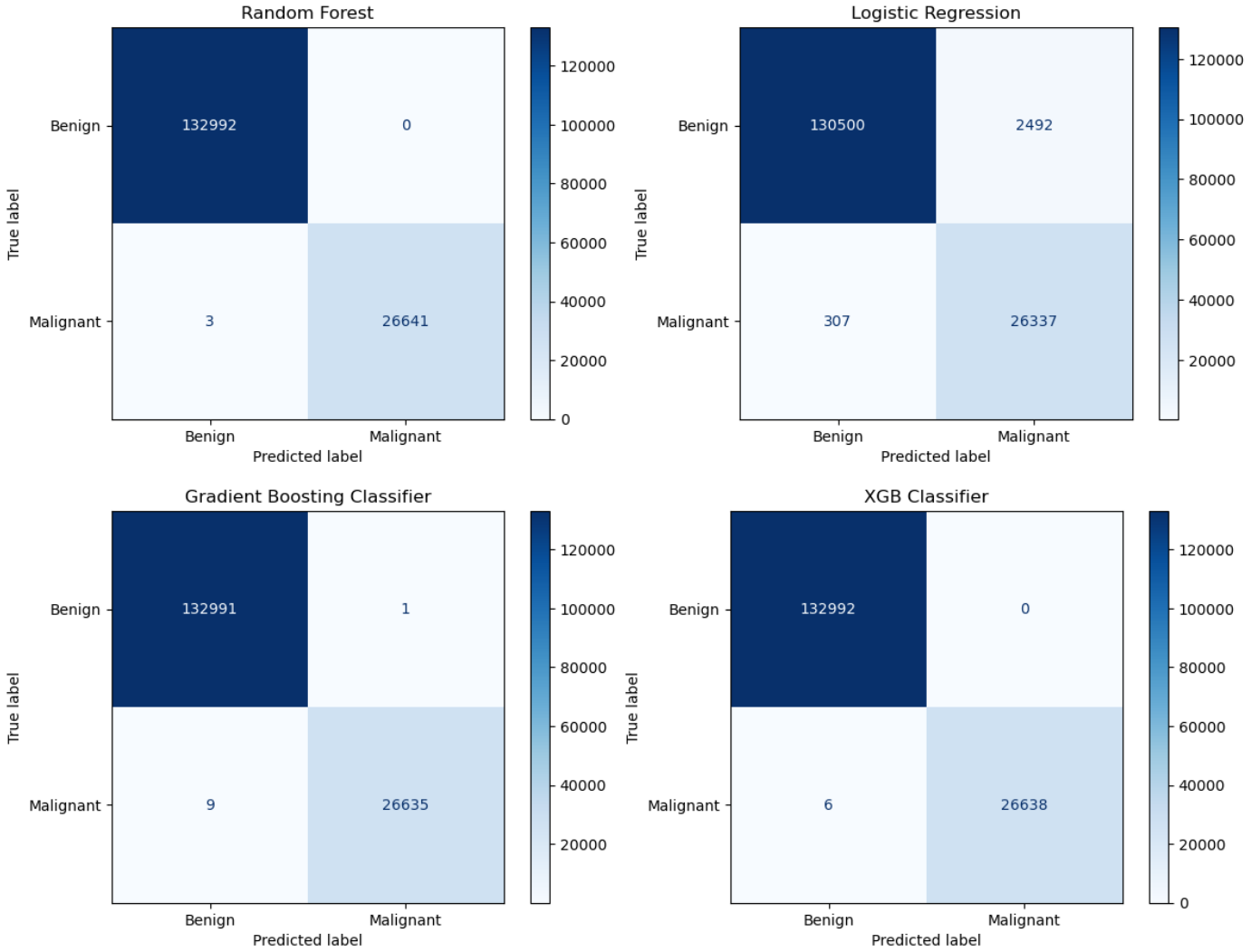
5.2 Model Training

Two models are trained to test the performance of the data. With poor accuracy (40%) and precision (21%), Gaussian Navie Bayes (GNB) will not be selected for further training. Decision Tree (DT) performs a perfect 1 accuracy, precision and recall, because DT only uses one tree to decide the result. It may cause high bias. Therefore, other ensemble learning models using DT as a foundation learning algorithm, such as Random Forest (RT), Gradient Boosting Classifier (GB) and XGB Classifier (XGB) are also trained to test the performance to see if any model has more robustness and accuracy. Logistic Regression Classifier (LR) is also trained as another type of classifier for comparison.

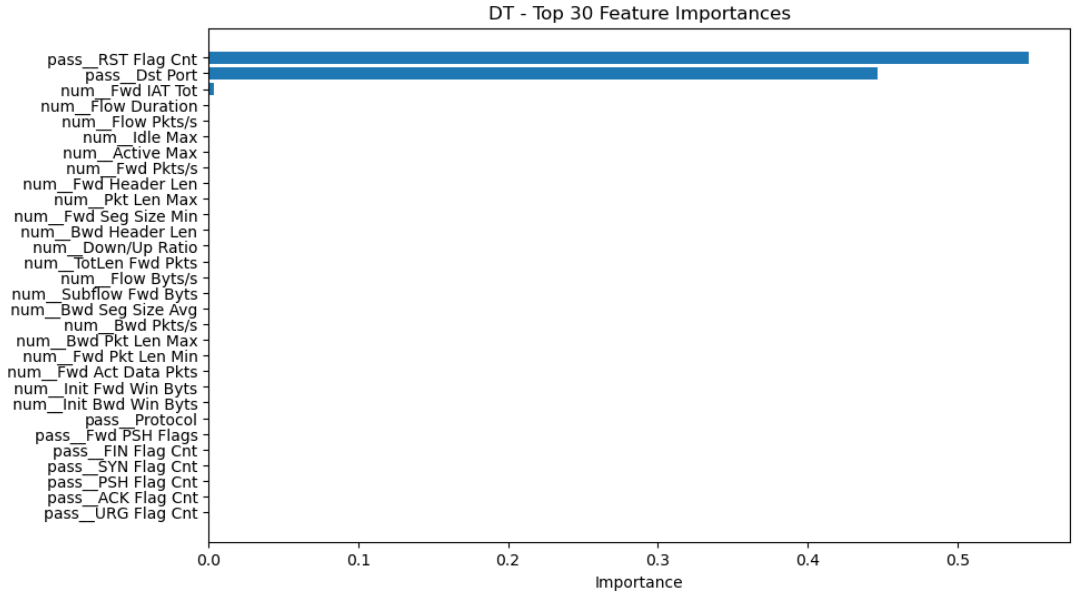
The result demonstrates that GNB receive low accuracy and precision because its characteristic of the Naïve assumption, which conditional independence among features. As spam has a specific behavioural network footprint similar to phishing and botnet - elevated flow rates, anomalous flag combinations, and short-lived connections, GNB do not apply to correlated features. However, because of these correlated features, other 5 models perform excellently in training. Especially for DT and other ensemble models, these models approach 1 in accuracy, precision and recall. As DT and other ensemble models use a hierarchical where each internal node represents a test on a feature, each branch represents the outcome, and each leaf node provides a final prediction., which is very suitable for classifying the spam according to its specific behaviour.

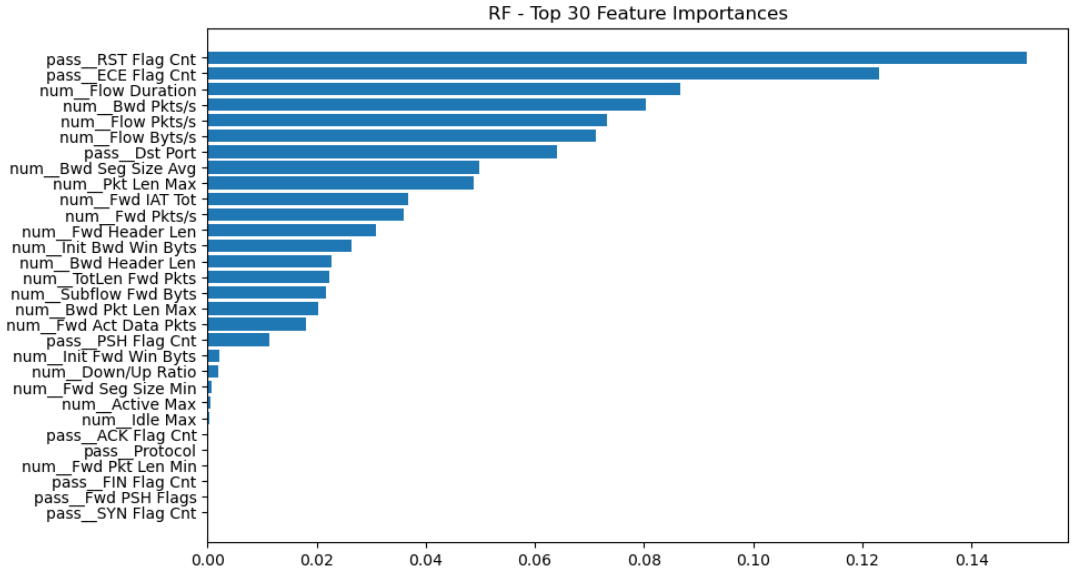
Models	Accuracy	Precision	Recall	F1-score
Gaussian Naïve Bayes	0.4090	0.2158	0.9649	0.3528
Decision Tree	1.0000	0.9999	0.9999	0.9999
Random Forest	1.0000	1.0000	0.9999	0.9999
Logistic Regression	0.9825	0.9136	0.9885	0.9495
Gradient Boosting Classifier	0.9999	1.0000	0.9997	0.9998
XGB Classifier	1.0000	1.0000	0.9998	0.9999

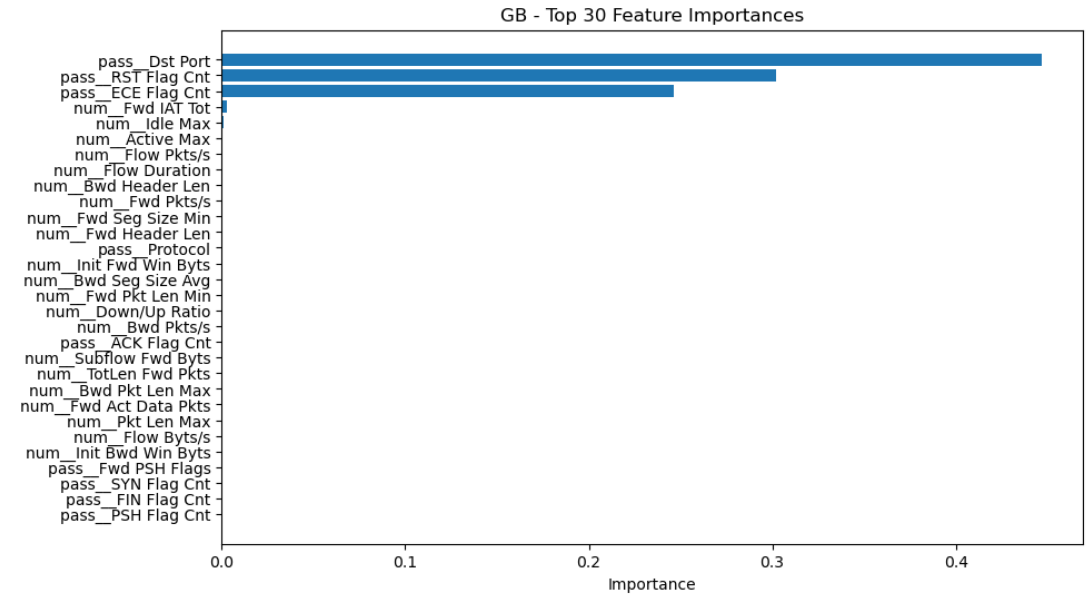


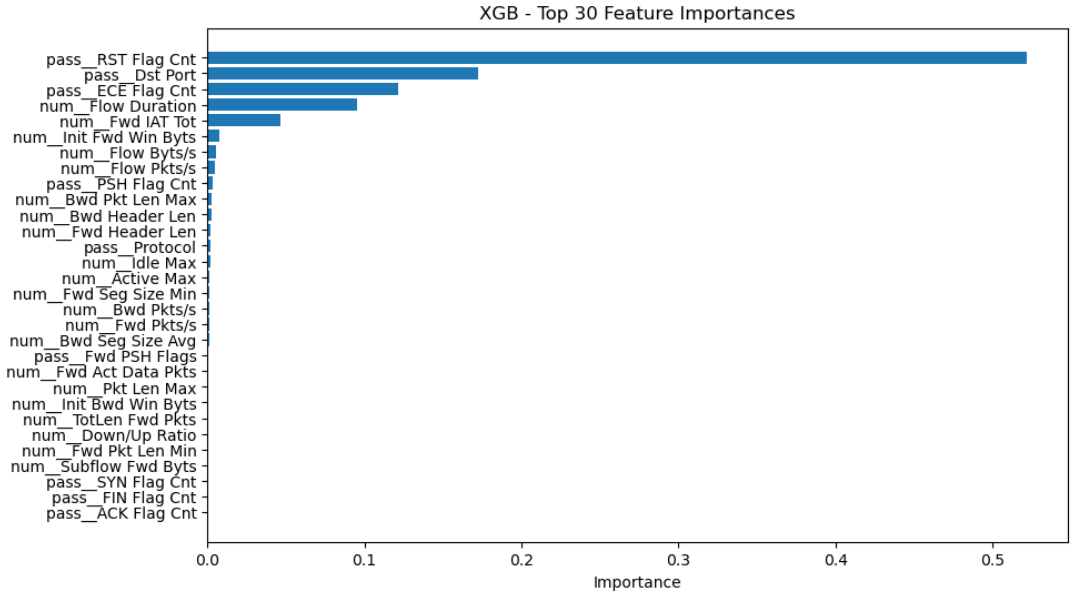


While DT and ensemble models perform remarkably well, to prevent any feature that extremely affects the decision of mode, top 30 feature importance of each classifier is also processed to analysis. Two features - 'pass_ECE Flag Cnt' and 'pass_Dst_Port' have high feature importance to all models, which shows they intervene models' decision obviously. Meanwhile, all models except RF, have 2 or more features' features importance are higher than 0.4. However, most of the features of RF are less than 0.15, which mean no specific features affect the decision of RF. Therefore, RF is selected for further training




RF - Top 30 Feature Importances
pass__RST Flag Cnt
pass__ECE Flag Cnt
num__Flow Duration
num__Bwd Pkts/s
num__Flow Pkts/s
num__Flow Byts/s
pass__Dst Port
num__Bwd Seg Size Avg
num__Pkt Len Max
num__Fwd IAT Tot
num__Fwd Pkts/s
num__Fwd Header Len
num__Init Bwd Win Byts
num__Bwd Header Len
num__TotLen Fwd Pkts
num__Subflow Fwd Byts
num__Bwd Pkt Len Max
num__Fwd Act Data Pkts
pass__PSH Flag Cnt
num__Init Fwd Win Byts
num__Down/Up Ratio
num__Fwd Seg Size Min
num__Active Max
num__Idle Max
pass__ACK Flag Cnt
pass__Protocol
num__Fwd Pkt Len Min
pass__FIN Flag Cnt
pass__Fwd PSH Flags
pass__SYN Flag Cnt
0.00
0.02
0.04
0.06
0.08
0.10
0.12
0.14

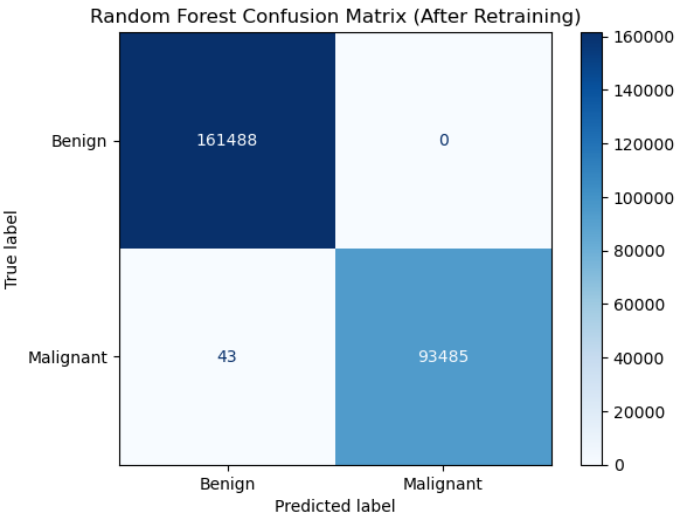

GB - Top 30 Feature Importances
pass__Dst Port
pass__RST Flag Cnt
pass__ECE Flag Cnt
num__Fwd IAT Tot
num__Idle Max
num__Active Max
num__Flow Pkts/s
num__Flow Duration
num__Bwd Header Len
num__Fwd Pkts/s
num__Fwd Seg Size Min
num__Fwd Header Len
pass__Protocol
num__Init Fwd Win Byts
num__Bwd Seg Size Avg
num__Fwd Pkt Len Min
num__Down/Up Ratio
num__Bwd Pkts/s
pass__ACK Flag Cnt
num__Subflow Fwd Byts
num__TotLen Fwd Pkts
num__Bwd Pkt Len Max
num__Fwd Act Data Pkts
num__Pkt Len Max
num__Flow Byts/s
num__Init Bwd Win Byts
pass__Fwd PSH Flags
pass__SYN Flag Cnt
pass__FIN Flag Cnt
pass__PSH Flag Cnt
0.0
0.1
0.2
0.3
0.4
Importance


XGB - Top 30 Feature Importances
pass__RST Flag Cnt
pass__Dst Port
pass__ECE Flag Cnt
num__Flow Duration
num__Fwd IAT Tot
num__Init Fwd Win Byts
num__Flow Byts/s
num__Flow Pkts/s
pass__PSH Flag Cnt
num__Bwd Pkt Len Max
num__Bwd Header Len
num__Fwd Header Len
pass__Protocol
num__Idle Max
num__Active Max
num__Fwd Seg Size Min
num__Bwd Pkts/s
num__Fwd Pkts/s
num__Bwd Seg Size Avg
pass__Fwd PSH Flags
num__Fwd Act Data Pkts
num__Pkt Len Max
num__Init Bwd Win Byts
num__TotLen Fwd Pkts
num__Down/Up Ratio
num__Fwd Pkt Len Min
num__Subflow Fwd Byts
pass__SYN Flag Cnt
pass__FIN Flag Cnt
pass__ACK Flag Cnt
0.0
0.1
0.2
0.3
0.4
0.5
Importance

Although RF has exemplary accuracy, precision and recall, it is a concern for predicting other unseen data. To access RF model's performance and robustness, cross validation is used to optimize its performance and avoid overfitting.

```
Start 5-fold Cross Validation...
Random Forest 5-fold CV Accuracy: 1.0000 (± 0.0000)
Model Score: 0.9999812072464858
Spam / Malignant Probability (first 10): [0. 1. 0. 0. 0. 0. 0. 0. 0. 1.]
```

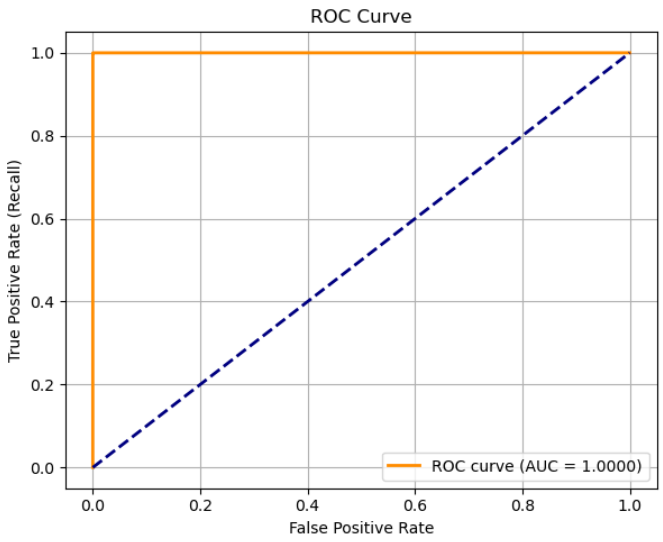
While RT performs well after 5-fold cross validation, RT is only trained from the botnet attack dataset. To maximize its performance on predicting malicious attack, the remaining DoS attack and brute force attack datasets from CSE-CIC-IDS2018 are used to upgrade the model. 02-14-2018.csv(brute force), 02-15-2018.csv(DoS), 02-16-2018.csv(DoS), 02-21-2018.csv(DoS), 02-22-2018.csv(brute force), 02-23-2018.csv(brute force) datasets are merged together, with all malignant data and 20% benign data from each dataset. The total rows of the merged dataset are 2,637,425 rows, including 1,723,919 benign and 913,506 malignant. After RF upgrade by the merged dataset, accuracy and precision drop a bit but it is within an acceptable range. The confidence of the spam prediction also drops from very confident to not much confidence, which shows RF learn to differentiate different attack patterns.

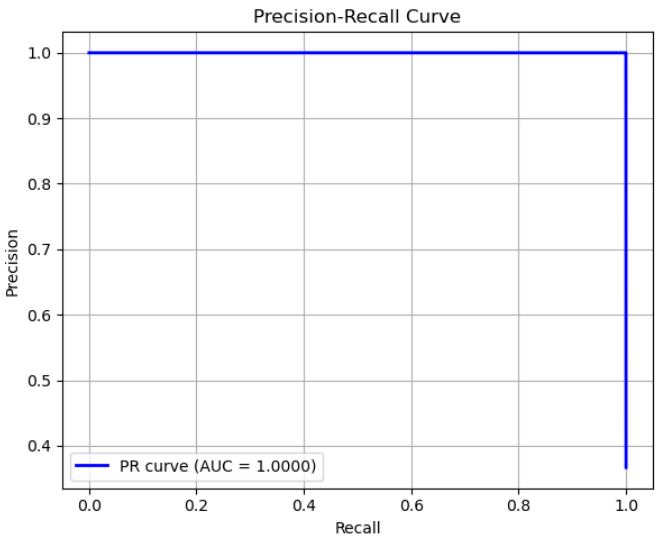


Random Forest (After Upgrading)				
Class / Metric	**Precision**	**Recall**	**F1-score**	**Support**
Benign	0.9996	1.0000	0.9998	161488
Malignant	1.0000	0.9994	0.9997	93528
Accuracy			0.9998	255016
Macro avg	0.9998	0.9997	0.9998	255016
Weighted avg	0.9998	0.9998	0.9998	255016

After upgrading the RF, cross validation is performed again. The accuracy is 0.9999. The ROC curve, PR curve and 1 AUC of RF show its false positive rate is extremely low and outstanding discrimination.

```
Updated Model Score: 0.9997921699030649
Spam / Malignant Probability (first 10): [0.79666667 0.         0.66666667 0.         0.75666667 0.76
 0.01       0.         0.         0.67       ]
Start 5-fold Cross Validation...
Random Forest 5-fold CV Accuracy: 0.9999 (± 0.0000)
```





6. Testing Plan & Results

To facilitate a stable and consistent virtual environment for IPS, Infrastructure as Code (IaC) tool – Terraform is used in this project. IaC is a method to configure and provision computing infrastructure with code rather than manual setup, which manual setup is time-consuming and prone to error. IaC can manage and maintain infrastructure components in a rapid and repeatable way. The following Ubuntu virtual server with t3.micro components are built in AWS.

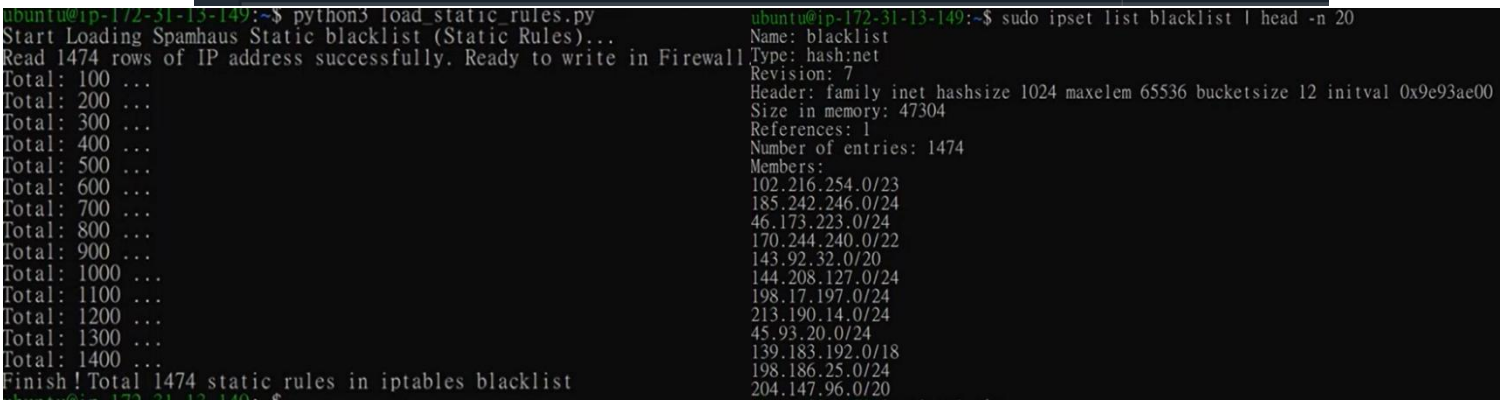
```
# ============================================
# main.tf – instantiate EC2 firewall (t2.micro + Ubuntu)
# ============================================

# Provide basic information(TF.file and AWS provider)
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# Connect to AWS
provider "aws" {
  region = "ap-east-1"   # Hong Kong Region
}

# Auto update to newest image: Ubuntu 22.04 LTS
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]   # Ubuntu Offical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
  }
  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

}

# Instantiate EC2
resource "aws_instance" "fyp_firewall" {
  ami           = data.aws_ami.ubuntu.id     # newest Ubuntu
  instance_type = "t3.micro"                 # free tier instance
  key_name      = "fyp-key"                 # Key-pair name

  # Storage setting
  root_block_device {
    volume_size           = 16
    volume_type           = "gp3"
    delete_on_termination = true             # delete harddisk on termination
  }

  # Instance name in AWS Console
  tags = {
    Name = "fyp_firewall"
  }

  # Auto public IP (for SSH)
  associate_public_ip_address = true
}
```

After initiating the virtual environment (EC2) for the IPS, it can connect through SSH. As the EC2 is new, python, ipset, scikit-learn and joblib library are required to install and update. The following files: drop_v4.csv, ips_monitor.py, load_static_rules.py, RF_spam_filter_350trees.pkl and traffic_sample.csv uploaded to the virtual machine through SCP for simulation attack.

To apply static rules in the IPS, blacklist and whitelist are created by ipset. The whitelist sets to accept all the packets in the list, and the blacklist sets to drop all the packets to port 25 in the list.

The python script below loads the Spamhaus drop_v4.csv IP list for static rules. After loading the drop_v4.csv and python script in the EC2, IP address in the file load into the blacklist for the static rules and show the total number of blocked IP addresses.

```python
import pandas as pd
import os

print("Start Loading Spamhaus Static blacklist (Static Rules)...")

# Reading drop_v4.csv
try:
    df = pd.read_csv('drop_v4.csv')

    # Change it to list
    bad_ips = df.iloc[:, 0].dropna().tolist()

    print(f"Read {len(bad_ips)} rows of IP address successfully. Ready to write in Firewall...")
except Exception as e:
    print(f"Fail to read CSV: {e}")
    exit()

# Confirm blacklist is built
os.system("sudo ipset create blacklist hash:net -exist")

#  Add IP to blacklist
count = 0
for ip in bad_ips:
    #  Ignore non IP format data
    if isinstance(ip, str) and ('.' in ip or ':' in ip):
        command = f"sudo ipset add blacklist {ip} -exist"
        os.system(command + " > /dev/null 2>&1")
        count += 1
        if count % 100 == 0:
            print(f"Total: {count} ...")

print(f"Finish ! Total {count} static rules in iptables blacklist")
```



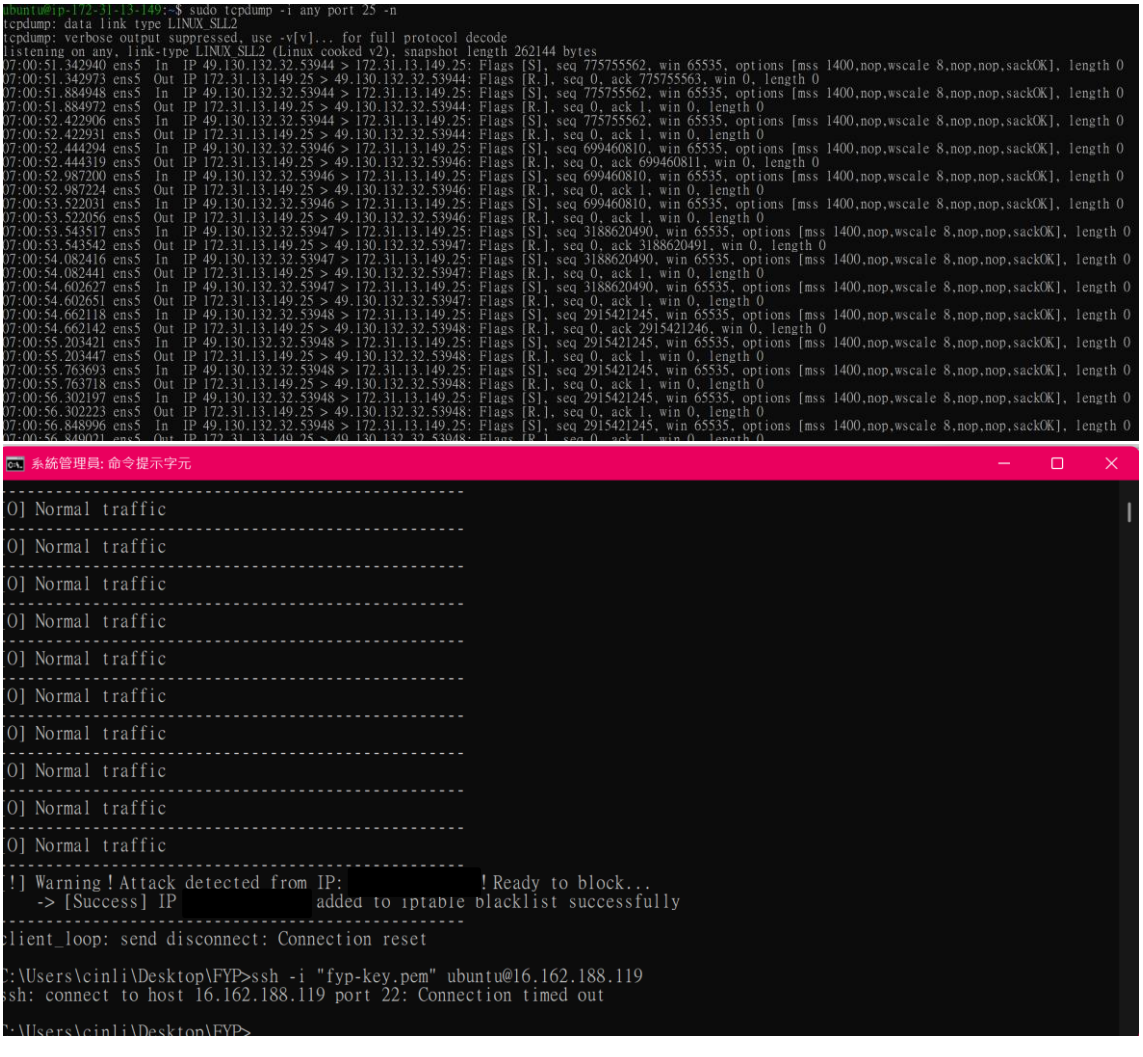
The python script below loads the model file: RF_spam_filter_350trees.pkl and use model to check the network flow. If any prediction is malicious, it directly commands to block the IP address.

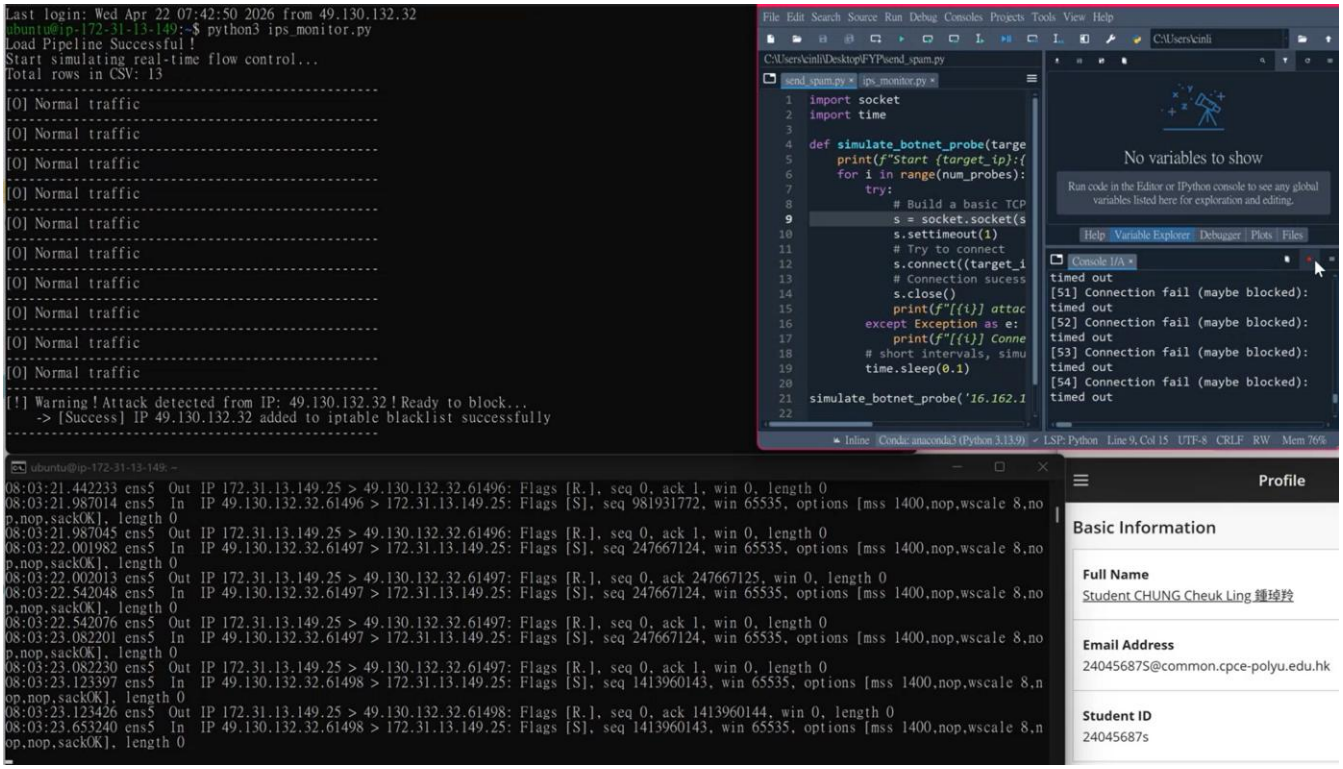
```python
import joblib
import pandas as pd
import time
import os

# --- Local Com Public IP ---
MY_REAL_ATTACKER_IP = '[redacted]'
# ----------------------------------------

# Load model
try:
    pipeline = joblib.load('RF_spam_filter_350trees.pkl')
    print("Load Pipeline Successful！")
except FileNotFoundError:
    print("Can not find RF_spam_filter_350trees.pkl, please check if it is in the folder.")
    exit()

# Reading testing flow data
try:
    test_data = pd.read_csv('traffic_sample.csv')
except FileNotFoundError:
    print("Can not find traffic_sample.csv。")
    exit()

# Define features for model
features_to_use = [
    'Dst Port', 'Protocol', 'Flow Duration', 'TotLen Fwd Pkts',
    'Fwd Pkt Len Min', 'Bwd Pkt Len Max', 'Flow Byts/s', 'Flow Pkts/s',
    'Fwd IAT Tot', 'Fwd PSH Flags', 'Fwd Header Len', 'Bwd Header Len',
    'Fwd Pkts/s', 'Bwd Pkts/s', 'Pkt Len Max', 'FIN Flag Cnt',
    'SYN Flag Cnt', 'RST Flag Cnt', 'PSH Flag Cnt', 'ACK Flag Cnt',
    'URG Flag Cnt', 'ECE Flag Cnt', 'Down/Up Ratio', 'Bwd Seg Size Avg',
    'Subflow Fwd Byts', 'Init Fwd Win Byts', 'Init Bwd Win Byts',
    'Fwd Act Data Pkts', 'Fwd Seg Size Min', 'Active Max', 'Idle Max'
]

print("Start simulating real-time flow control...")
print(f"Total rows in CSV: {len(test_data)}")
print("-" * 50)

# Simulating to read every flow and predict
for i in range(len(test_data)):
    # Get feature data
    single_flow_features = test_data[features_to_use].iloc[[i]]

    # Prediction
    prediction = pipeline.predict(single_flow_features)

    # Trigger
    if prediction[0] == 1: # Malignant
        print(f"[!] Warning！Attack detected from IP: {MY_REAL_ATTACKER_IP}！Ready to block...")

        # os.system call Linux command, block {MY_REAL_ATTACKER_IP} to blacklist
        block_command = f"sudo ipset add blacklist {MY_REAL_ATTACKER_IP}"

        # Run command and hidden system default message
        result = os.system(block_command + " > /dev/null 2>&1")

        if result == 0:
            print(f"    -> [Success] IP {MY_REAL_ATTACKER_IP} added to iptable blacklist successfully")
        else:
            print(f"    -> [Info] IP {MY_REAL_ATTACKER_IP} maybe in blacklist already or invalid privileges。")
    else:
        print(f"[O] Normal traffic")

    print("-" * 50)
    time.sleep(2)
```

The python script below tests our model through network layer simulation attack. The script mainly focuses on port 25 and sends short intervals packages, simulating high-frequency scanning.

```python
import socket
import time

def simulate_botnet_probe(target_ip, target_port=25, num_probes=1000):
    print(f"Start {target_ip}:{target_port} Similating Botnet detection...")
    for i in range(num_probes):
        try:
            # Build a basic TCP Socket
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            s.settimeout(1)
            # Try to connect
            s.connect((target_ip, target_port))
            # Connection sucess, do not send any L7 data, close connection
            s.close()
            print(f"[{i}] attack package sent!")
        except Exception as e:
            print(f"[{i}] Connection fail (maybe blocked): {e}")
        # short intervals, simulating high-frequency scanning
        time.sleep(0.1)

simulate_botnet_probe('16.162.188.119', target_port=25, num_probes=1000)
```

Below screen captures show the tcpdump of simulation attack while attacker is attacking EC2's port 25. As the EC2 does not open port 25, it sends reject [R] back. When RF predicts it is an attack, it blocks and includes the IP address to the blacklist. After blocking the IP address, despite the attacker still sending packets, tcpdump does not have any update. The attacker's IP is listed in blacklist successfully.



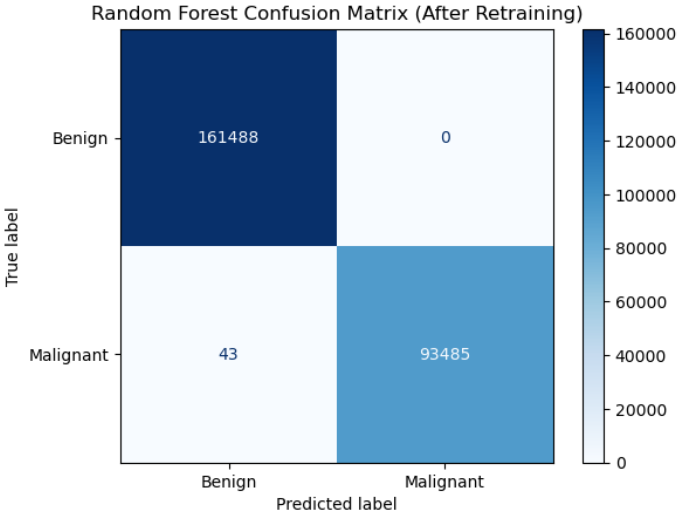


7. Evaluation

The proposed dynamic IPS demonstrated excellent results in the controlled simulation attacks conducted on the AWS EC2 Ubuntu server. The hybrid system combining static blacklist/whitelist rules with dynamic rules generated by the Random Forest model. It successfully blocked malicious IP addresses in real-time during high-frequency port-25 scanning tests. The dynamic ML-enhanced rules significantly outperformed static rules alone, delivering high blocking effectiveness with minimal latency and low resource consumption on the t3.micro instance.

7.1 Model Performance Analysis

To provide a deeper understanding of the metrics, the performance of the final Random Forest model is examined through its confusion matrix. The matrix reveals near-perfect classification on the merged CSE-CIC-IDS2018 test set:



True Positive (TP): 161,488 benign flows correctly allowed (100 % recall for legitimate traffic).

False Negative (FN): Only 43 malicious flows missed (extremely low, indicating excellent detection of spam/botnet traffic).

True Negative (TN): 93,485 malicious flows correctly identified as attacks (99.94 % of all malignant samples).

False Positive (FP): 0 benign flows incorrectly flagged (0.00 %).

For a firewall/IPS deployment, False Positive rate is the most critical metric. As an excessively high FP would result in blocking legitimate users, causing service disruption and user frustration. The Random Forest model's extremely low FP rate (only 0 out of 161,488 benign flows) confirms its suitability for production-grade network protection, striking an outstanding balance between security and usability.

While achieving an accuracy of 1.00 may typically raise concerns of overfitting, this is a valid characteristic of the spam attack network behavioural footprint. Automated botnets exhibit highly rigid and deterministic "physics patterns" at the TCP/IP layer, such as fixed packet lengths, machine-like flow durations, and repetitive flag combinations. The Random Forest model successfully captured these non-human behavioral signatures, distinguishing them perfectly from benign traffic.

For completeness, the project also compared multiple classifiers as originally proposed. Gaussian Naïve Bayes was ultimately not selected because network flow features are highly correlated, and do not satisfy the algorithm's naïve independence assumption or Gaussian distribution requirement, resulting in poor accuracy (0.4090) and precision (0.2158). In contrast, tree-based ensemble methods (especially Random Forest) naturally handle multicollinearity and non-linear interactions, delivering superior and robust performance.

7.2 System Mitigation Effectiveness

The real-world effectiveness of the hybrid IPS was validated through live simulation attacks using the traffic_sample.csv script targeting port 25.

Before mitigation (static rules only): The attack script successfully sent high-frequency packets. The tcpdump capture showed continuous incoming SYN packets and the server responding with TCP Reset [R.] packets, indicating that the server was actively suffering TCP connection exhaustion at the network layer.

After enabling the dynamic ML component (ips_monitor.py): Once the Random Forest model classified a flow as malicious, the Python script instantly executed an ipset command to add the attacker's IP to the blacklist. Subsequent tcpdump output became completely silent: no further packets from the attacker reached the server.

The system demonstrated near-instantaneous mitigation. Once the ML pipeline classified a flow as malicious, the Python script executed the ipset command in milliseconds, effectively halting the SYN flood at the network edge. This confirms that the dynamic rule generation mechanism works seamlessly with the underlying firewall.

7.3 System Limitations & Architectural Trade-offs

Despite the simulation results, several practical limitations must be acknowledged. A major architectural trade-off in this project was the method of real-time feature extraction. Extracting the full set of 31 complex statistical features (such as Flow IAT Tot, Active Max, etc.) from raw PCAP streams in real-time requires deep packet inspection and significant computational resources, which exceeds the capacity of a standard AWS t3.micro instance. Therefore, the evaluation adopted a data-injection/simulation approach: a pre-processed traffic stream (CSV) was used to simulate the output of a hardware-level flow meter. This approach successfully validated the core objective — proving that the ML-driven decision engine can seamlessly integrate with the OS-level firewall (iptables/ipset) for dynamic threat isolation. For a production-ready environment, integrating a lightweight, real-time flow exporter (such as Zeek or CICFlowMeter-V3) directly into the EC2 instance would be the necessary next step.

This study successfully demonstrated the viability of using machine learning to generate dynamic firewall rules based on network-level features. The model was trained and validated using the well-

established CSE-CIC-IDS-2018 benchmark dataset. A potential limitation is the age of the dataset, as adversary tactics evolve over time. For a real-world production system, it would be crucial to implement a continuous learning pipeline, periodically retraining the model on more recent, site-specific network traffic to ensure its continued efficacy against emerging threats.

Additional limitations include the manual configuration of the whitelist and the inherent variability of real-world encrypted traffic that was not fully replicated in the simulation. Nevertheless, the privacy-preserving, flow-level design and lightweight implementation make the solution highly suitable for resource-constrained environments.

8. Conclusion

This project has successfully developed and prototyped a Dynamic Intrusion Prevention System (IPS) that integrates machine learning-generated dynamic rules with traditional static blacklist and whitelist mechanisms to mitigate spam traffic at the network flow level. Leveraging the CSE-CIC-IDS2018 dataset and rigorous feature engineering, the Random Forest classifier achieved near-perfect performance metrics (accuracy, precision, recall, and F1-score all approaching 1.0). The hybrid IPS, deployed on an AWS EC2 Ubuntu server using Terraform Infrastructure as Code, demonstrated in simulation tests that dynamic ML rules significantly outperform static rules alone while maintaining efficiency and preserving user privacy.

The two research questions formulated in the literature review have been fully addressed. First, key network-level features, such as destination port, flow duration, packet rates, inter-arrival times, and TCP flag counts were confirmed as strong predictors of spam-related malicious traffic without the need for content inspection. Second, the dynamic ML-generated rule set provided a clear and measurable improvement in spam-blocking effectiveness compared with a massive static blacklist.

The project moved from dataset processing and model selection to full system implementation and real-time testing. The resulting lightweight, interpretable, and privacy-preserving solution demonstrates strong potential for practical deployment in small-to-medium organizations or edge environments.

While limitations such as dataset age and the need for continuous learning exist, this work establishes a solid foundation for adaptive network security. Future enhancements could include automated whitelist management, integration of online learning mechanisms, incorporation of more recent datasets, and live-traffic evaluation. Overall, the Dynamic IPS developed in this project illustrates the practical value of combining traditional security controls with machine learning, paving the way for more intelligent and responsive defense against evolving spam and botnet threats.

9. Reference

Ahmed, N., Amin, R., Aldabbas, H., Koundal, D., Alouffi, B., & Shaq, T. (2022). *Machine Learning Techniques for Spam Detection in Email and IoT Platforms: Analysis and Research Challenges* (Volume 2022, Issue 1). Security and Communication Networks. https://onlinelibrary.wiley.com/doi/10.1155/2022/1862888

Bayrak, S., Karaca, A., Toson, F., Tayfur, M. E., & Yavas, S. (2025). Unified AI Models for Network Security on Edge Devices. *International Journal of Computational Intelligence Systems, 18*(1), 266. https://doi.org/10.1007/s44196-025-00990-6

CloudHQ. (2026). *Email statistics report 2025-2030 [Updated 2026]*. https://blog.cloudhq.net/email-statistics-report-2025-2030/

Communications Security Establishment, & Canadian Institute for Cybersecurity. (n.d.). *CSE-CIC-IDS2018 on AWS*. University of New Brunswick. https://www.unb.ca/cic/datasets/ids-2018.html

Dada, E. G., Bassi, J. S., Chiroma, H., Abdulhamid, S. M., Adetunmbi, A. O., & Ajibuwa, O. E. (2019). Machine learning for email spam filtering: Review, approaches and open research problems. *Heliyon*, *5*(6), e01802. https://doi.org/10.1016/j.heliyon.2019.e01802

De Santis, A., Castiglione, A., Fiore, U., & Palmieri, F. (2013). An intelligent security architecture for distributed firewalling environments. *Journal of Ambient Intelligence and Humanized Computing, 4*(2), 223-234. https://doi.org/10.1007/s12652-011-0069-8

Moutafis, I., Andreatos, A., & Stefaneas, P. (2023). *Spam Email Detection Using Machine Learning Techniques*. Academic Conferences International Limited.

Niu, Y., Chen, C., Zhang, X., Zhou, X., & Liu, H. (2022). Application of a New Feature Generation Algorithm in Intrusion Detection System. *Wireless Communications & Mobile Computing (Online), 2022*https://doi.org/10.1155/2022/3794579

Olivo, C., Santin, A., Viegas, E., Geremias, J., & Souto, E. (2024, October 21). Towards a reliable spam detection: An ensemble classification with rejection option (Vol. 28, Iss. 1). Cluster

Computing. https://doi.org/10.1007/s10586-024-04742-7

Su, S. (2025). Research on spam filters based on NB algorithm. *ITM Web of Conferences*, *70*, 01016. https://doi.org/10.1051/itmconf/20257001016

Yang, C. (2019). Anomaly network traffic detection algorithm based on information entropy measurement under the cloud computing environment. *Cluster Computing, Suppl.4, 22*, 8309-8317. https://doi.org/10.1007/s10586-018-1755-5

Zhang, C. (2025). *Enhancing Spam Filtering: A Comparative Study of Modern Advanced Machine Learning Techniques*. EDP Sciences. https://doi.org/10.1051/itmconf/20257004013

Zhang, K., Wang, H., Chen, M., Chen, X., Liu, L., Geng, Q., & Zhou, Y. (2025). Leveraging machine learning to proactively identify phishing campaigns before they strike. *Journal of Big Data*, *12*(1). https://doi.org/10.1186/s40537-025-01174-x

10. Appendix

Appendix 1

CSE-CIC-IDS2018 datasets comparison

The following chart shows details and the malicious percentage in network traffic of each dataset:

Dataset	Total Data	Malicious	Benign	Malicious %
02-14-2018.csv	1,048,575	193,360 (FTP-Brute Force) 187,589 (SSH-Brute Force)	667,626	~36%
02-15-2018.csv	1,048,575	41,508 (DoS attacks-GoldenEye) 10,990 (DoS attacks-Slowloris)	996,077	~5%
02-16-2018.csv	1,048,575	461,912 (DoS attacks-Hulk) 139,890 (DoS attacks-SlowHTTPTest)	446,772	~57%
02-20-2018.csv	7,948,748	576,191 (DDoS attacks-LOIC-HTTP)	7,372,557	~7%
02-21-2018.csv	1,048,575	686,012 (DDOS attack-HOIC) 1,730 (DDOS attack-LOIC-UDP)	360,833	~66%
02-22-2018.csv	1,048,575	249 (Brute Force -Web) 79 (Brute Force -XSS) 34 (SQL Injection)	1,048,213	~0%
02-23-2018.csv	1,048,575	362 (Brute Force -Web) 151 (Brute Force -XSS) 53 (SQL Injection)	1,048,009	~0%
02-28-2018.csv	613,104	68,871 (Infiltration)	544,200	~11%
03-01-2018.csv	331,125	93,063 (Infiltration)	238,037	~28%
03-02-2018.csv	1,048,575	286,191 (Bot)	762,384	~27%

Appendix 2

CSE-CIC-IDS2018 dataset features detail:

No.	Feature Name	Description
1.	Dst Port	Destination port of connection.
2.	Protocol	Protocol used during connection.
3.	Timestamp	Time that connection occured.
4.	fl_dur	Flow duration
5.	tot_fw_pk	Total packets in the forward direction
6.	tot_bw_pk	Total packets in the backward direction
7.	tot_l_fw_pkt	Total size of packet in forward direction
8.	tot_b_fw_pkt	Total size of packet in backward direction
9.	fw_pkt_l_max	Maximum size of packet in forward direction
10.	fw_pkt_l_min	Minimum size of packet in forward direction
11.	fw_pkt_l_avg	Average size of packet in forward direction
12.	fw_pkt_l_std	Standard deviation size of packet in forward direction
13.	Bw_pkt_l_max	Maximum size of packet in backward direction
14.	Bw_pkt_l_min	Minimum size of packet in backward direction
15.	Bw_pkt_l_avg	Mean size of packet in backward direction
16.	Bw_pkt_l_std	Standard deviation size of packet in backward direction
17.	fl_byt_s	flow byte rate that is number of packets transferred per second
18.	fl_pkt_s	flow packets rate that is number of packets transferred per second
19.	fl_iat_avg	Average time between two flows
20.	fl_iat_std	Standard deviation time two flows
21.	fl_iat_max	Maximum time between two flows
22.	fl_iat_min	Minimum time between two flows
23.	fw_iat_tot	Total time between two packets sent in the forward direction
24.	fw_iat_avg	Mean time between two packets sent in the forward direction
25.	fw_iat_std	Standard deviation time between two packets sent in the forward direction
26.	fw_iat_max	Maximum time between two packets sent in the forward direction
27.	fw_iat_min	Minimum time between two packets sent in the forward direction
28.	bw_iat_tot	Total time between two packets sent in the backward direction
29.	bw_iat_avg	Mean time between two packets sent in the backward direction
30.	bw_iat_std	Standard deviation time between two packets sent in the backward direction
31.	bw_iat_max	Maximum time between two packets sent in the backward direction
32.	bw_iat_min	Minimum time between two packets sent in the backward direction
33.	fw_psh_flag	Number of times the PSH flag was set in packets travelling in the forward direction (0 for UDP)
34.	bw_psh_flag	Number of times the PSH flag was set in packets travelling in the backward direction (0 for UDP)
35.	fw_urg_flag	Number of times the URG flag was set in packets travelling in the forward direction (0 for UDP)
36.	bw_urg_flag	Number of times the URG flag was set in packets travelling in the backward direction (0 for UDP)
37.	fw_hdr_len	Total bytes used for headers in the forward direction
38.	bw_hdr_len	Total bytes used for headers in the forward direction
39.	fw_pkt_s	Number of forward packets per second

40.	bw_pkt_s	Number of backward packets per second
41.	pkt_len_min	Minimum length of a flow
42.	pkt_len_max	Maximum length of a flow
43.	pkt_len_avg	Mean length of a flow
44.	pkt_len_std	Standard deviation length of a flow
45.	pkt_len_va	Minimum inter-arrival time of packet
46.	fin_cnt	Number of packets with FIN
47.	syn_cnt	Number of packets with SYN
48.	rst_cnt	Number of packets with RST
49.	pst_cnt	Number of packets with PUSH
50.	ack_cnt	Number of packets with ACK
51.	urg_cnt	Number of packets with URG
52.	cwe_cnt	Number of packets with CWE
53.	ece_cnt	Number of packets with ECE
54.	down_up_ratio	Download and upload ratio
55.	pkt_size_avg	Average size of packet
56.	fw_seg_avg	Average size observed in the forward direction
57.	bw_seg_avg	Average size observed in the backward direction
58.	fw_byt_blk_avg	Average number of bytes bulk rate in the forward direction
59.	fw_pkt_blk_avg	Average number of packets bulk rate in the forward direction
60.	fw_blk_rate_avg	Average number of bulk rate in the forward direction
61.	bw_byt_blk_avg	Average number of bytes bulk rate in the backward direction
62.	bw_pkt_blk_avg	Average number of packets bulk rate in the backward direction
63.	bw_blk_rate_avg	Average number of bulk rate in the backward direction
64.	subfl_fw_pk	The average number of packets in a sub flow in the forward direction
65.	subfl_fw_byt	The average number of bytes in a sub flow in the forward direction
66.	subfl_bw_pkt	The average number of packets in a sub flow in the backward direction
67.	subfl_bw_byt	The average number of bytes in a sub flow in the backward direction
68.	fw_win_byt	Number of bytes sent in initial window in the forward direction
69.	bw_win_byt	# of bytes sent in initial window in the backward direction
70.	Fw_act_pkt	# of packets with at least 1 byte of TCP data payload in the forward direction
71.	fw_seg_min	Minimum segment size observed in the forward direction
72.	atv_avg	Mean time a flow was active before becoming idle
73.	atv_std	Standard deviation time a flow was active before becoming idle
74.	atv_max	Maximum time a flow was active before becoming idle
75.	atv_min	Minimum time a flow was active before becoming idle
76.	idl_avg	Mean time a flow was idle before becoming active
77.	idl_std	Standard deviation time a flow was idle before becoming active
78.	idl_max	Maximum time a flow was idle before becoming active
79.	idl_min	Minimum time a flow was idle before becoming active

Appendix 3

Select K-Best result

Feature	Score (f_classif)
pass__RST Flag Cnt	414554.005507
pass__ECE Flag Cnt	414554.005507
pass__PSH Flag Cnt	102588.873645
num__Bwd Pkt Len Max	51326.716078
num__Init Bwd Win Byts	41148.716761
num__Bwd Seg Size Avg	35415.702192
num__Down/Up Ratio	34819.526263
pass__ACK Flag Cnt	34059.364420
num__Fwd Seg Size Min	30382.684777
pass__Protocol	28327.927460
num__Flow Duration	25572.102552
num__Fwd IAT Tot	24490.374971
num__Fwd Pkt Len Min	19123.455488
num__Pkt Len Max	12849.731951
num__Idle Max	12682.213461
pass__URG Flag Cnt	5271.231256
pass__SYN Flag Cnt	4154.789682
pass__Fwd PSH Flags	4154.789682
num__Flow Pkts/s	1745.896257
num__Fwd Pkts/s	1524.276667
num__Fwd Act Data Pkts	1152.498571
num__TotLen Fwd Pkts	1060.008573
num__Subflow Fwd Byts	1060.008573
num__Bwd Pkts/s	971.254639
num__Active Max	810.730561
pass__FIN Flag Cnt	590.434145
num__Flow Byts/s	307.030783
num__Init Fwd Win Byts	117.626358
num__Fwd Header Len	49.087625
num__Bwd Header Len	47.562472

Appendix 4

Features for model training

No.	Feature Name	Description	Category
1.	Dst Port	Destination port of connection.	
2.	Protocol	Protocol used during connection.	
3.	fl_dur	Flow duration	Volume Feature
4.	tot_l_fw_pkt	Total size of packet in forward direction	Volume Feature
5.	fw_pkt_l_min	Minimum size of packet in forward direction	Shape Features
6.	Bw_pkt_l_max	Maximum size of packet in backward direction	Shape Features
7.	fl_byt_s	flow byte rate that is number of packets transferred per second	Volume Feature
8.	fl_pkt_s	flow packets rate that is number of packets transferred per second	Volume Feature
9.	fw_iat_tot	Total time between two packets sent in the forward direction	Behavioral Features
10.	fw_psh_flag	Number of times the PSH flag was set in packets travelling in the forward direction (0 for UDP)	Behavioral Features
11.	fw_hdr_len	Total bytes used for headers in the forward direction	Shape Features
12.	bw_hdr_len	Total bytes used for headers in the forward direction	Shape Features
13.	fw_pkt_s	Number of forward packets per second	Volume Feature
14.	bw_pkt_s	Number of backward packets per second	Volume Feature
15.	pkt_len_max	Maximum length of a flow	Shape Features
16.	fin_cnt	Number of packets with FIN	Behavioral Features
17.	syn_cnt	Number of packets with SYN	Behavioral Features
18.	rst_cnt	Number of packets with RST	Behavioral Features
19.	psh_cnt	Number of packets with PUSH	Behavioral Features
20.	ack_cnt	Number of packets with ACK	Behavioral Features
21.	urg_cnt	Number of packets with URG	Behavioral Features
22.	ece_cnt	Number of packets with ECE	Behavioral Features
23.	down_up_ratio	Download and upload ratio	Behavioral Features
24.	bw_seg_avg	Average size observed in the backward direction	Shape Features
25.	subfl_fw_byt	The average number of bytes in a sub flow in the forward direction	Shape Features
26.	fw_win_byt	Number of bytes sent in initial window in the forward direction	Behavioral Features
27.	bw_win_byt	# of bytes sent in initial window in the backward direction	Behavioral Features
28.	Fw_act_pkt	# of packets with at least 1 byte of TCP data payload in the forward direction	Volume Feature
29.	fw_seg_min	Minimum segment size observed in the forward direction	Shape Features
30.	atv_max	Maximum time a flow was active before becoming idle	Shape Features
31.	idl_max	Maximum time a flow was idle before becoming active	Shape Features

Appendix 5

Result of all models

Gaussian Naïve Bayes

Class / Metric	Precision	Recall	F1-score	Support
Benign	0.98	0.30	0.46	132992
Malignant	0.22	0.96	0.35	26644
Accuracy			0.41	159636
Macro avg	0.60	0.63	0.40	159636
Weighted avg	0.85	0.41	0.44	159636

Decision Tree

Class / Metric	Precision	Recall	F1-score	Support
Benign	1.00	1.00	1.00	132992
Malignant	1.00	1.00	1.00	26644
Accuracy			1.00	159636
Macro avg	1.00	1.00	1.00	159636
Weighted avg	1.00	1.00	1.00	159636

Random Forest

Class / Metric	Precision	Recall	F1-score	Support
Benign	1.00	1.00	1.00	132992
Malignant	1.00	1.00	1.00	26644
Accuracy			1.00	159636
Macro avg	1.00	1.00	1.00	159636
Weighted avg	1.00	1.00	1.00	159636

Logistic Regression

Class / Metric	Precision	Recall	F1-score	Support
Benign	1.00	0.98	0.99	132992
Malignant	0.91	0.99	0.95	26644
Accuracy			0.98	159636
Macro avg	0.96	0.98	0.97	159636
Weighted avg	0.98	0.98	0.98	159636

Gradient Boosting Classifier

Class / Metric	Precision	Recall	F1-score	Support
Benign	1.00	1.00	1.00	132992
Malignant	1.00	1.00	1.00	26644
Accuracy			1.00	159636
Macro avg	1.00	1.00	1.00	159636
Weighted avg	1.00	1.00	1.00	159636

XGB Classifier

Class / Metric	Precision	Recall	F1-score	Support
Benign	1.00	1.00	1.00	132992
Malignant	1.00	1.00	1.00	26644
Accuracy			1.00	159636
Macro avg	1.00	1.00	1.00	159636
Weighted avg	1.00	1.00	1.00	159636

Appendix C – Supervision Log Sheet *(to be filled in by students)*

Student could download this log sheet in word format on my.SPEED.

Meeting No.	Date & Time	Duration & Venue	Discussion items / Agreed tasks and action plans for the next meeting
1	06/02/2026 15:42	30 minutes, WK-S1311	**Discussion items:** Student: Proposed the topic of developing a Network Firewall Rule Set to Block Spam Traffic (Machine Learning Rule Generation) Show secondary dataset is going to use Briefly explain the objects, research question, timeline and skillset of the project Supervisor: The topic is fine if the logic of the project makes sense. The role of the project needs to be defined. Need to consider how to execute, how to reduce the misjudgment or exception of the email **Agreed tasks and action plans:** Student needs to define the role of the project. Consider thoughtfully how to define spam by network layer, whether there is a reply or else, how the dataset trains the model. More research and literature review is needed.
2	13/03/2026 14:45	30 minutes, WK-S1208	**Discussion items:** Student: Update 2 objectives about literature review and result analysis. Update changes from the proposal Report on the project process: • Terraform script • Dataset selection • Feature engineering Query details about progress report, final report and presentation Supervisor: So far so good. Try SelectK-best algorithm for the feature engineering Prepare videos for the presentation if needed

			Agreed tasks and action plans: Try SelectK-best algorithm for the feature engineering Prepare videos for the presentation if needed
3	8/4/2026 16:00	30 minutes, WK-S1311	**Discussion items:** Student: Update the project process: • Model selection • Each model training result • Overfitting issue Query details about final report and presentation Supervisor: It is normal for the models have "perfect 1" accuracy and precision as spam have its specific network physical pattern. In addition, models' performance is affected by the large size of dataset. It is fine to use the model and there is no overfitting for the models. **Agreed tasks and action plans:** Use the model for further simulation attack.
4 5	23/4/2026 16:15	45 minutes WK-S1204	**Discussion items:** Student: Present the project: • Project background • Feature engineering and Model selection • Each model training result • Retrain model result • Show simulation attack Supervisor: So far so good.

Student Name:	CHUNG CHEUK LING	Student Signature:	
Supervisor Name:	Dr Adam Wong	Supervisor Signature:	

*Note: This supervision log sheet must be submitted together with the **Final Report**.*